

CS1200 / CSCI E-120 Lecture 7

The RAM Model, RAM Simulations, and the Word-RAM Model

Fall 2026

Announcements

- Midterm grades out.
- Pset 3 due tomorrow.
- No in-class reflection for pset 3 (due to Indigenous People's Day and SRE on next Wed).
- Pset 4 out on Wed.
- Hesterberg-Vadhan: 6.1,6.2,7.1,7.2, 8.1.

Computational Models

- Sorting: compare keys, update pointers.

Computational Models

- Sorting: compare keys, update pointers.
- EstimateProportions: read any element of array in one step.

Computational Models

- Sorting: compare keys, update pointers.
- EstimateProportions: read any element of array in one step.
- QuickSelect: choose a random pivot.

Computational Models

- Sorting: compare keys, update pointers.
- EstimateProportions: read any element of array in one step.
- QuickSelect: choose a random pivot.
- ...
- What are the limitations? What is allowed?

Computational Models

- Sorting: compare keys, update pointers.
- EstimateProportions: read any element of array in one step.
- QuickSelect: choose a random pivot.
- ...
- What are the limitations? What is allowed?
- Computational models make this precise and give us a ‘platform’ to work on. They also specify a ‘complexity measure’ to compare algorithms (formalizing the ‘runtime’ we have been using so far).

Computational Models

Q: What do we want from a computational model and a complexity measure?

- **Technological relevance.**

Computational Models

Q: What do we want from a computational model and a complexity measure?

- **Technological relevance.** The model and its complexity measure should reasonably capture the implementability and efficiency of algorithms on actual hardware, even as technology evolves. Even better if the model is inspired by widely used hardware framework.
- **Unambiguity.**

Computational Models

Q: What do we want from a computational model and a complexity measure?

- **Technological relevance.** The model and its complexity measure should reasonably capture the implementability and efficiency of algorithms on actual hardware, even as technology evolves. Even better if the model is inspired by widely used hardware framework.
- **Unambiguity.** Precisely define algorithms (syntax and semantics) and their runtime, including what counts as a basic operation.

Computational Models

Q: What do we want from a computational model and a complexity measure?

- **Technological relevance.** The model and its complexity measure should reasonably capture the implementability and efficiency of algorithms on actual hardware, even as technology evolves. Even better if the model is inspired by widely used hardware framework.
- **Unambiguity.** Precisely define algorithms (syntax and semantics) and their runtime, including what counts as a basic operation.
- **Mathematical simplicity.**

Computational Models

Q: What do we want from a computational model and a complexity measure?

- **Technological relevance.** The model and its complexity measure should reasonably capture the implementability and efficiency of algorithms on actual hardware, even as technology evolves. Even better if the model is inspired by widely used hardware framework.
- **Unambiguity.** Precisely define algorithms (syntax and semantics) and their runtime, including what counts as a basic operation.
- **Mathematical simplicity.** Make the model manageable to reason about and prove things about.

Computational Models

Desiderata (continued)

- Expressivity.

Computational Models

Desiderata (continued)

- **Expressivity.** Express everything we intuitively consider an algorithm, or give a good reason for excluding it.

Computational Models

Desiderata (continued)

- **Expressivity.** Express everything we intuitively consider an algorithm, or give a good reason for excluding it.
- **Robustness.**

Computational Models

Desiderata (continued)

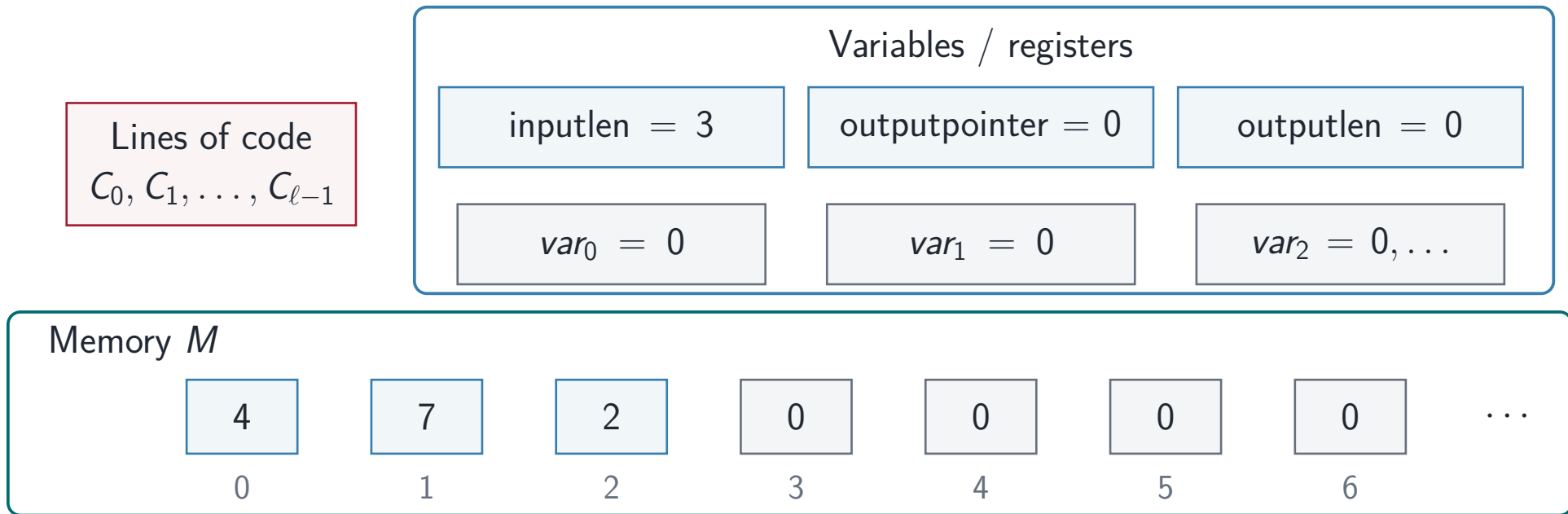
- **Expressivity.** Express everything we intuitively consider an algorithm, or give a good reason for excluding it.
- **Robustness.** Small changes to the model should not fundamentally change what can be computed, and should have only a small effect on efficiency.

RAM programs - definition

Memory is an infinite array M of natural numbers. A RAM program $P = (\mathcal{V}, C_0, \dots, C_{\ell-1})$ has a finite set $\mathcal{V}$ of variables (registers) and a finite sequence of commands.

RAM programs - definition

Memory is an infinite array M of natural numbers. A RAM program $P = (\mathcal{V}, C_0, \dots, C_{\ell-1})$ has a finite set $\mathcal{V}$ of variables (registers) and a finite sequence of commands.



Every program includes the special variables `inputlen`, `outputpointer`, and `outputlen`.

RAM programs - definition

For registers $var_0, var_1, var_2 \in \mathcal{V}$, the allowed commands are:

- Constant assignment: $var_0 = d$, where $d \in \mathbb{N}$.

RAM programs - definition

For registers $var_0, var_1, var_2 \in \mathcal{V}$, the allowed commands are:

- Constant assignment: $var_0 = d$, where $d \in \mathbb{N}$.
- Arithmetic: $var_1 = var_2 \text{ op } var_3$, where $\text{op} \in \{+, -, \times, /\}$.

RAM programs - definition

For registers $var_0, var_1, var_2 \in \mathcal{V}$, the allowed commands are:

- Constant assignment: $var_0 = d$, where $d \in \mathbb{N}$.
- Arithmetic: $var_1 = var_2 \text{ op } var_3$, where $\text{op} \in \{+, -, \times, /\}$.
- Read from memory: $var_0 = M[var_1]$; write: $M[var_0] = var_1$.

RAM programs - definition

For registers $var_0, var_1, var_2 \in \mathcal{V}$, the allowed commands are:

- Constant assignment: $var_0 = d$, where $d \in \mathbb{N}$.
- Arithmetic: $var_1 = var_2 \text{ op } var_3$, where $\text{op} \in \{+, -, \times, /\}$.
- Read from memory: $var_0 = M[var_1]$; write: $M[var_0] = var_1$.
- Conditional jump: IF $var_0 = 0$, GOTO Line k , where $0 \leq k \leq \ell$.

Semantics of RAM program

Computation of a RAM Program

1. Initialization.

Semantics of RAM program

Computation of a RAM Program

1. **Initialization.** ‘Load’ the input x as $(M[0], \dots, M[n - 1])$. Every remaining memory cell is zero. Set $\text{inputlen} = n$ and every other variable to zero.

Semantics of RAM program

Computation of a RAM Program

1. **Initialization.** ‘Load’ the input x as $(M[0], \dots, M[n-1])$. Every remaining memory cell is zero. Set $\text{inputlen} = n$ and every other variable to zero.
2. **Execution.**

Semantics of RAM program

Computation of a RAM Program

1. **Initialization.** ‘Load’ the input x as $(M[0], \dots, M[n - 1])$. Every remaining memory cell is zero. Set $\text{inputlen} = n$ and every other variable to zero.
2. **Execution.** Execute commands in order, except for jumps, updating registers and memory. Subtraction is clipped at zero; division rounds down; division by zero returns zero.

Semantics of RAM program

Computation of a RAM Program

1. **Initialization.** ‘Load’ the input x as $(M[0], \dots, M[n-1])$. Every remaining memory cell is zero. Set $\text{inputlen} = n$ and every other variable to zero.
2. **Execution.** Execute commands in order, except for jumps, updating registers and memory. Subtraction is clipped at zero; division rounds down; division by zero returns zero.

$$3 - 5 = \quad , \quad 7/2 = \quad , \quad 7/0 = \quad .$$

Semantics of RAM program

Computation of a RAM Program

1. **Initialization.** ‘Load’ the input x as $(M[0], \dots, M[n-1])$. Every remaining memory cell is zero. Set $\text{inputlen} = n$ and every other variable to zero.
2. **Execution.** Execute commands in order, except for jumps, updating registers and memory. Subtraction is clipped at zero; division rounds down; division by zero returns zero.

$$3 - 5 = 0, \quad 7/2 = 3, \quad 7/0 = 0.$$

The RAM Model

Definition: Computation of a RAM Program—semantics (continued)

3. Output.

The RAM Model

Definition: Computation of a RAM Program—semantics (continued)

3. Output. If execution reaches line ℓ , including via a jump, the program halts. Its output is the memory slice of length `outputlen` starting at `outputpointer`:

$$P(x) = (M[\text{outputpointer}], \dots, M[\text{outputpointer} + \text{outputlen} - 1]).$$

The RAM Model

Definition: Computation of a RAM Program—semantics (continued)

3. Output. If execution reaches line ℓ , including via a jump, the program halts. Its output is the memory slice of length `outputlen` starting at `outputpointer`:

$$P(x) = (M[\text{outputpointer}], \dots, M[\text{outputpointer} + \text{outputlen} - 1]).$$

Running time.

The RAM Model

Definition: Computation of a RAM Program—semantics (continued)

3. Output. If execution reaches line ℓ , including via a jump, the program halts. Its output is the memory slice of length `outputlen` starting at `outputpointer`:

$$P(x) = (M[\text{outputpointer}], \dots, M[\text{outputpointer} + \text{outputlen} - 1]).$$

Running time. $\text{Time}_P(x)$ is the number of commands executed on input x . It may be ∞ if the program does not halt.

Unambiguity and mathematical simplicity

- The definitions specify the syntax, semantics, and running time precisely.

Unambiguity and mathematical simplicity

- The definitions specify the syntax, semantics, and running time precisely.
- The model has a short mathematical description, compared with the detailed specifications of modern programming languages.

Unambiguity and mathematical simplicity

- The definitions specify the syntax, semantics, and running time precisely.
- The model has a short mathematical description, compared with the detailed specifications of modern programming languages.

Q: What are other models out there?

Unambiguity and mathematical simplicity

- The definitions specify the syntax, semantics, and running time precisely.
- The model has a short mathematical description, compared with the detailed specifications of modern programming languages.

Q: What are other models out there?

C, Java, Python, SQL, Turing Machines, Lambda Calculus, etc...

Unambiguity and mathematical simplicity

- The definitions specify the syntax, semantics, and running time precisely.
- The model has a short mathematical description, compared with the detailed specifications of modern programming languages.

Q: What are other models out there?

C, Java, Python, SQL, Turing Machines, Lambda Calculus, etc...

Turing Machines are even simpler to define, but do not represent real hardware.

Expressiveness: Simulating High-Level Programs

Theorem (informal)

1. Every Python program (similarly, C, Java, etc.) can be simulated by a RAM program.

Expressiveness: Simulating High-Level Programs

Theorem (informal)

1. Every Python program (similarly, C, Java, etc.) can be simulated by a RAM program.
2. Every RAM program can be simulated by a Python program (similarly, C, Java, etc.).

Expressiveness: Simulating High-Level Programs

Theorem (informal)

1. Every Python program (similarly, C, Java, etc.) can be simulated by a RAM program.
2. Every RAM program can be simulated by a Python program (similarly, C, Java, etc.).

Q: What do we mean by “simulation”?

Expressiveness: Simulating High-Level Programs

Theorem (informal)

1. Every Python program (similarly, C, Java, etc.) can be simulated by a RAM program.
2. Every RAM program can be simulated by a Python program (similarly, C, Java, etc.).

Q: What do we mean by “simulation”?

For every program P in model $\mathcal{M}$, there is a program Q in model $\mathcal{N}$ with the same behavior on every input x :

- $P(x)$ and $Q(x)$ produce the same output

Expressiveness: Simulating High-Level Programs

Theorem (informal)

1. Every Python program (similarly, C, Java, etc.) can be simulated by a RAM program.
2. Every RAM program can be simulated by a Python program (similarly, C, Java, etc.).

Q: What do we mean by “simulation”?

For every program P in model $\mathcal{M}$, there is a program Q in model $\mathcal{N}$ with the same behavior on every input x :

- $P(x)$ and $Q(x)$ produce the same output if they halt.
- $P(x)$ halts if and only if $Q(x)$ halts.

Expressiveness: Simulating High-Level Programs

Proof sketch: high-level programs to RAM

Expressiveness: Simulating High-Level Programs

Proof sketch: high-level programs to RAM

- Compilers translate high-level programs into assembly language, which is translated into machine code.

Expressiveness: Simulating High-Level Programs

Proof sketch: high-level programs to RAM

- Compilers translate high-level programs into assembly language, which is translated into machine code.
- RAM closely resembles assembly language: its variables correspond to registers, and it can simulate the instructions.

Expressiveness: Simulating High-Level Programs

Proof sketch: high-level programs to RAM

- Compilers translate high-level programs into assembly language, which is translated into machine code.
- RAM closely resembles assembly language: its variables correspond to registers, and it can simulate the instructions.

Proof sketch: RAM programs to Python Pset 4! You'll construct a universal simulator:

Expressiveness: Simulating High-Level Programs

Proof sketch: high-level programs to RAM

- Compilers translate high-level programs into assembly language, which is translated into machine code.
- RAM closely resembles assembly language: its variables correspond to registers, and it can simulate the instructions.

Proof sketch: RAM programs to Python Pset 4! You'll construct a universal simulator:
A **universal simulator** U takes both the program and its input:

$$U(P, x) \text{ halts iff } P(x) \text{ halts,} \quad U(P, x) = P(x) \text{ when they halt.}$$

Expressiveness: Simulating High-Level Programs

Proof sketch: high-level programs to RAM

- Compilers translate high-level programs into assembly language, which is translated into machine code.
- RAM closely resembles assembly language: its variables correspond to registers, and it can simulate the instructions.

Proof sketch: RAM programs to Python Pset 4! You'll construct a universal simulator:
A **universal simulator** U takes both the program and its input:

$$U(P, x) \text{ halts iff } P(x) \text{ halts,} \quad U(P, x) = P(x) \text{ when they halt.}$$

Hardcoding P gives an ordinary simulator $Q(x) = U(P, x)$.

Robustness

Small changes to the RAM model do not change the power.

Theorem: simulating an additional operation

The *mod-extended RAM model* allows a modulo operation $var_0 = var_1 \% var_2$. Every program P in this model can be simulated by a standard RAM program Q , with

$$\text{Time}_Q(x) \leq 3\text{Time}_P(x).$$

Robustness

Small changes to the RAM model do not change the power.

Theorem: simulating an additional operation

The *mod-extended RAM model* allows a modulo operation $var_0 = var_1 \% var_2$. Every program P in this model can be simulated by a standard RAM program Q , with

$$\text{Time}_Q(x) \leq 3\text{Time}_P(x).$$

Corollary. A problem is solvable in time $O(T(n))$ in the RAM model if and only if it is solvable in time $O(T(n))$ in the mod-extended RAM model.

Robustness

Proof

Replace $var_0 = var_1 \bmod var_2$ by the following three commands, using a fresh temporary *temp*:

Robustness

Proof

Replace $var_0 = var_1 \bmod var_2$ by the following three commands, using a fresh temporary $temp$:

1. $temp = var_1 / var_2$

Robustness

Proof

Replace $var_0 = var_1 \bmod var_2$ by the following three commands, using a fresh temporary $temp$:

1. $temp = var_1 / var_2$
2. $temp = temp \times var_2$

Robustness

Proof

Replace $var_0 = var_1 \bmod var_2$ by the following three commands, using a fresh temporary $temp$:

1. $temp = var_1 / var_2$
2. $temp = temp \times var_2$
3. $var_0 = var_1 - temp$

Robustness

Proof

Replace $var_0 = var_1 \bmod var_2$ by the following three commands, using a fresh temporary $temp$:

1. $temp = var_1 / var_2$
2. $temp = temp \times var_2$
3. $var_0 = var_1 - temp$

Update every GOTO target to the start of the corresponding translated block. Each old command becomes at most three commands, so $\text{Time}_Q(x) = O(\text{Time}_P(x))$.

Technological Relevance

RAM model should be able to simulate programs running on physical computers.

Two pushbacks:

1. **Number of registers.**

Technological Relevance

RAM model should be able to simulate programs running on physical computers.

Two pushbacks:

1. **Number of registers.** Hardware has a fixed number; a RAM program may use an arbitrary finite number of variables.

Technological Relevance

RAM model should be able to simulate programs running on physical computers.

Two pushbacks:

1. **Number of registers.** Hardware has a fixed number; a RAM program may use an arbitrary finite number of variables.
2. **Size of numbers.**

Technological Relevance

RAM model should be able to simulate programs running on physical computers.

Two pushbacks:

1. **Number of registers.** Hardware has a fixed number; a RAM program may use an arbitrary finite number of variables.
2. **Size of numbers.** Hardware registers and memory cells have a bounded word length; RAM stores arbitrary natural numbers.

Technological Relevance

RAM model should be able to simulate programs running on physical computers.

Two pushbacks:

1. **Number of registers.** Hardware has a fixed number; a RAM program may use an arbitrary finite number of variables.
2. **Size of numbers.** Hardware registers and memory cells have a bounded word length; RAM stores arbitrary natural numbers.

We address the first issue by simulation, and the second with Word-RAM.

Technological Relevance

Theorem: a fixed number of registers suffices

There is a fixed constant $c (\leq 8)$ such that every RAM program P can be simulated by a RAM program P' using at most c variables.

Technological Relevance

Theorem: a fixed number of registers suffices

There is a fixed constant $c (\leq 8)$ such that every RAM program P can be simulated by a RAM program P' using at most c variables. The construction gives

$$\text{Time}_{P'}(x) = O(\text{Time}_P(x) + |P(x)|).$$

Here $|P(x)|$ is the length of the output, measured in memory cells.

Technological Relevance

Theorem: a fixed number of registers suffices

There is a fixed constant $c (\leq 8)$ such that every RAM program P can be simulated by a RAM program P' using at most c variables. The construction gives

$$\text{Time}_{P'}(x) = O(\text{Time}_P(x) + |P(x)|).$$

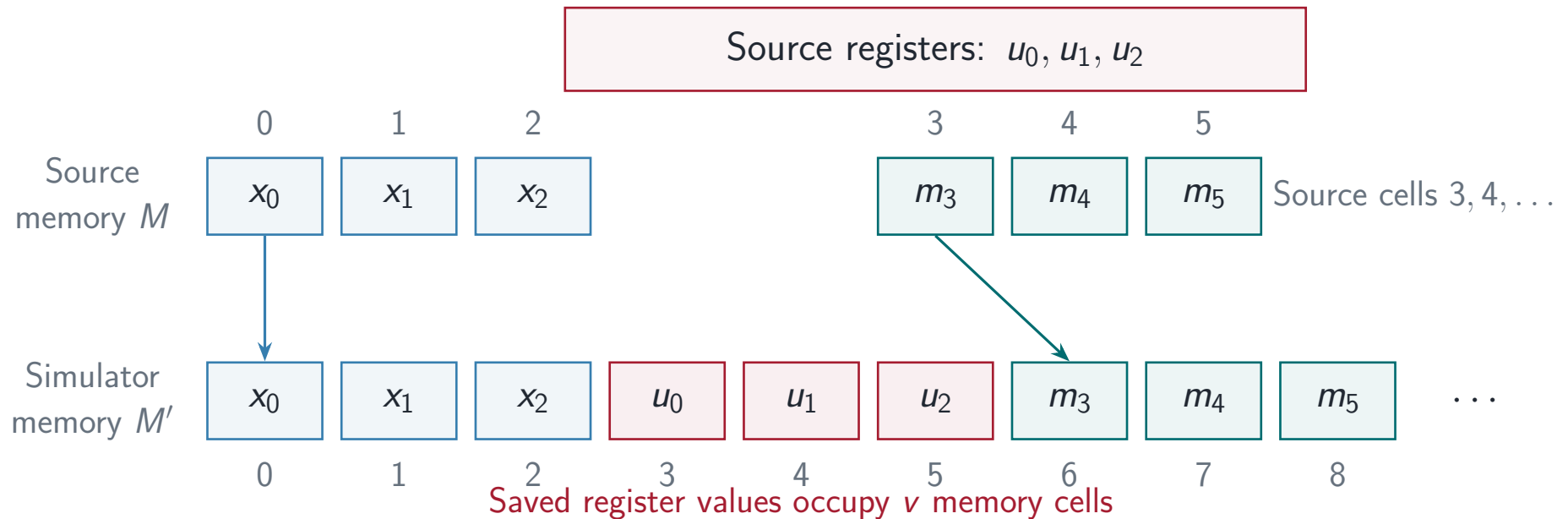
Here $|P(x)|$ is the length of the output, measured in memory cells.

Proof idea: store the variables in memory!

Technological Relevance

Call P as 'source', P' as 'simulator'.

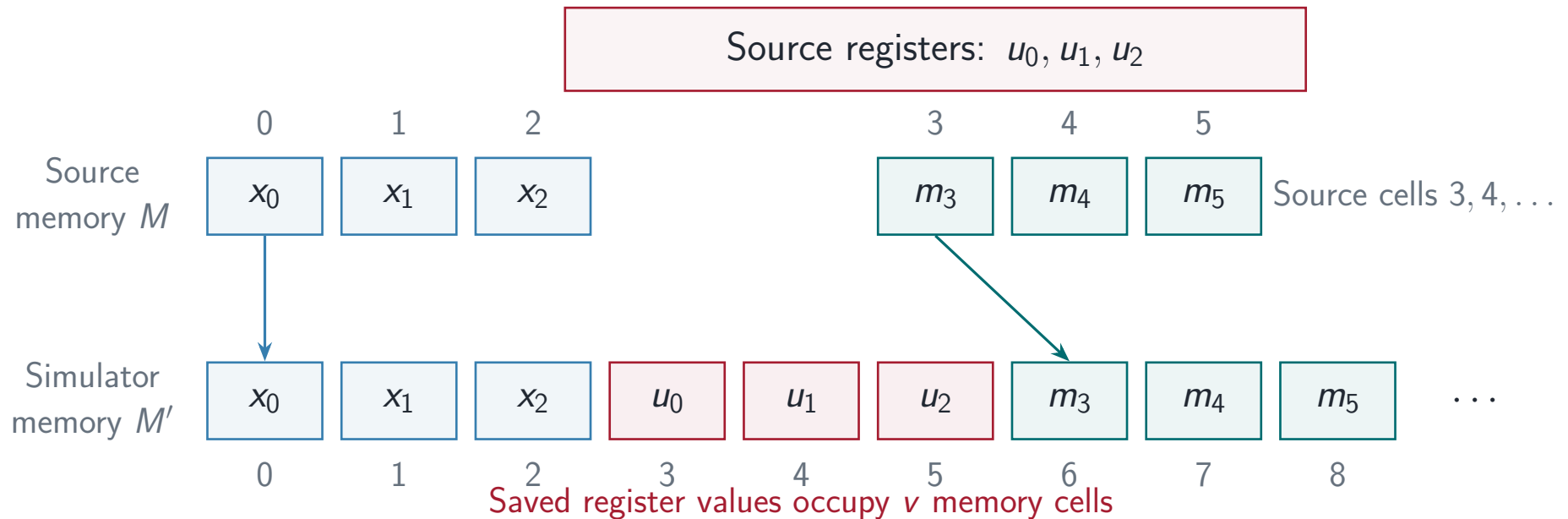
The memory layout: input length $n = 3$, source registers $v = 3$



Technological Relevance

Call P as 'source', P' as 'simulator'.

The memory layout: input length $n = 3$, source registers $v = 3$



P' itself uses only a fixed set of temporary registers and its three special registers.

Technological Relevance

Proof: a memory-read command

For a read $var_i = M[var_j]$:

Technological Relevance

Proof: a memory-read command

For a read $var_i = M[var_j]$:

1. Figure out var_j from $M'[n + j]$ and store into a temporary register $temp_2$ (need $temp_0, temp_1$ to compute $n + j$).
2. If $temp_2 \geq n$, add v to $temp_2$ to offset the 'variable zone'.
3. Read $temp_3 = M'[temp_2]$.
4. Write $temp_3$ to $M'[n + i]$, the location that saves var_i .

Technological Relevance

Proof: a memory-read command

For a read $var_i = M[var_j]$:

1. Figure out var_j from $M'[n + j]$ and store into a temporary register $temp_2$ (need $temp_0, temp_1$ to compute $n + j$).
2. If $temp_2 \geq n$, add v to $temp_2$ to offset the 'variable zone'.
3. Read $temp_3 = M'[temp_2]$.
4. Write $temp_3$ to $M'[n + i]$, the location that saves var_i .

Memory writes use the same address translation. Each case takes $O(1)$ commands and reuses the same temporary registers.

Technological Relevance

Proof: a memory-read command

For a read $var_i = M[var_j]$:

1. Figure out var_j from $M'[n + j]$ and store into a temporary register $temp_2$ (need $temp_0, temp_1$ to compute $n + j$).
2. If $temp_2 \geq n$, add v to $temp_2$ to offset the 'variable zone'.
3. Read $temp_3 = M'[temp_2]$.
4. Write $temp_3$ to $M'[n + i]$, the location that saves var_i .

Memory writes use the same address translation. Each case takes $O(1)$ commands and reuses the same temporary registers.

Similar argument applies to operations and GOTOs (see Section 7.3 in HV).

Technological Relevance

Q: What still goes wrong with the output?

Technological Relevance

Q: What still goes wrong with the output? For $n = 3$ and $v = 3$, suppose P returns output at the memory locations 1, 2, 3. The corresponding simulator locations are

$$(M[1], M[2], M[3]) \longmapsto (M'[1], M'[2], M'[6]).$$

Technological Relevance

Q: What still goes wrong with the output? For $n = 3$ and $v = 3$, suppose P returns output at the memory locations 1, 2, 3. The corresponding simulator locations are

$$(M[1], M[2], M[3]) \longmapsto (M'[1], M'[2], M'[6]).$$

These cells are separated by the saved registers.

Technological Relevance

Q: What still goes wrong with the output? For $n = 3$ and $v = 3$, suppose P returns output at the memory locations 1, 2, 3. The corresponding simulator locations are

$$(M[1], M[2], M[3]) \longmapsto (M'[1], M'[2], M'[6]).$$

These cells are separated by the saved registers.

Fix: Let n be the original input length and $L = |P(x)|$. Before applying the simulation, modify P so that its output starts at location n .

Technological Relevance

Q: What still goes wrong with the output? For $n = 3$ and $v = 3$, suppose P returns output at the memory locations 1, 2, 3. The corresponding simulator locations are

$$(M[1], M[2], M[3]) \longmapsto (M'[1], M'[2], M'[6]).$$

These cells are separated by the saved registers.

Fix: Let n be the original input length and $L = |P(x)|$. Before applying the simulation, modify P so that its output starts at location n . Append a loop that copies the output to

$$M[n], M[n + 1], \dots, M[n + L - 1]$$

and sets `outputpointer = n`.

Technological Relevance

Q: What still goes wrong with the output? For $n = 3$ and $v = 3$, suppose P returns output at the memory locations 1, 2, 3. The corresponding simulator locations are

$$(M[1], M[2], M[3]) \longmapsto (M'[1], M'[2], M'[6]).$$

These cells are separated by the saved registers.

Fix: Let n be the original input length and $L = |P(x)|$. Before applying the simulation, modify P so that its output starts at location n . Append a loop that copies the output to

$$M[n], M[n+1], \dots, M[n+L-1]$$

and sets $\text{outputpointer} = n$. This costs $O(L) = O(|P(x)|)$ additional steps. The output now lies entirely outside the original input region.

Technological Relevance

Proof: output and running time

At the end, read the saved values of `outputlen` and `outputpointer`.

Technological Relevance

Proof: output and running time

At the end, read the saved values of `outputlen` and `outputpointer`. Set $\text{outputlen}' = \text{outputlen}$ and $\text{outputpointer}' = \text{outputpointer} + v$.

Technological Relevance

Proof: output and running time

At the end, read the saved values of `outputlen` and `outputpointer`. Set $\text{outputlen}' = \text{outputlen}$ and $\text{outputpointer}' = \text{outputpointer} + v$.

The simulator uses $t_0, \dots, t_4$ and its three special registers: at most 8 in total.

$$\text{Time}_{P'}(x) = O(\text{Time}_P(x) + |P(x)|).$$

The Word-RAM Model

Word length and large integers

RAM permits arithmetic on arbitrarily large natural numbers in one step. Physical registers instead hold a bounded number of bits, called the *word length* w .

The Word-RAM Model

Word length and large integers

RAM permits arithmetic on arbitrarily large natural numbers in one step. Physical registers instead hold a bounded number of bits, called the *word length* w .

Q: How can we represent and compute on larger numbers (e.g. multiply two 1024-bit numbers)?

The Word-RAM Model

Word length and large integers

RAM permits arithmetic on arbitrarily large natural numbers in one step. Physical registers instead hold a bounded number of bits, called the *word length* w .

Q: How can we represent and compute on larger numbers (e.g. multiply two 1024-bit numbers)?

Use *bignum arithmetic*: store each number as an array of digits. An L -bit number has $m = \lceil L/w \rceil$ digits in base 2^w .

$$N = d_0 + d_1 2^w + \cdots + d_{m-1} 2^{w(m-1)}.$$

Bignum arithmetic: counting word operations

A product of two w -bit words may need $2w$ bits.

The Word-RAM Model

Definition: memory and allocation

A Word-RAM program is a RAM program with a *static word length* w and a *dynamic memory size* S .

The Word-RAM Model

Definition: memory and allocation

A Word-RAM program is a RAM program with a *static word length* w and a *dynamic memory size* S .

- Choose w before execution; it stays fixed during that run.
- Registers and memory cells hold values in $\{0, \dots, 2^w - 1\}$.
- Memory consists of cells $M[0], \dots, M[S - 1]$; initially $S = n$.

The Word-RAM Model

Definition: memory and allocation

A Word-RAM program is a RAM program with a *static word length* w and a *dynamic memory size* S .

- Choose w before execution; it stays fixed during that run.
- Registers and memory cells hold values in $\{0, \dots, 2^w - 1\}$.
- Memory consists of cells $M[0], \dots, M[S - 1]$; initially $S = n$.
- MALLOC increments S by one.
- Reads or writes at addresses $j \geq S$ have no effect.

The Word-RAM Model

Definition: output and arithmetic

Output. On halting, return the requested memory slice, truncated at the end of allocated memory:

$$(M[\text{outputpointer}], \dots, M[\min\{\text{outputpointer} + \text{outputlen} - 1, S - 1\}]).$$

The Word-RAM Model

Definition: output and arithmetic

Output. On halting, return the requested memory slice, truncated at the end of allocated memory:

$$(M[\text{outputpointer}], \dots, M[\min\{\text{outputpointer} + \text{outputlen} - 1, S - 1\}]).$$

Operations. Addition and multiplication return $2^w - 1$ whenever their exact result would be at least 2^w .

The Word-RAM Model

Definition: output and arithmetic

Output. On halting, return the requested memory slice, truncated at the end of allocated memory:

$$(M[\text{outputpointer}], \dots, M[\min\{\text{outputpointer} + \text{outputlen} - 1, S - 1\}]).$$

Operations. Addition and multiplication return $2^w - 1$ whenever their exact result would be at least 2^w .

This is *saturation arithmetic*. With $w = 4$,

$$12 + 7 = 15, \quad 12 \times 3 = 15.$$

Subtraction and division retain their RAM conventions.

The Word-RAM Model

Definition: crashing

A program P crashes on input x and word length w if:

The Word-RAM Model

Definition: crashing

A program P crashes on input x and word length w if:

1. A constant c in an assignment command of P satisfies $c \geq 2^w$.

The Word-RAM Model

Definition: crashing

A program P crashes on input x and word length w if:

1. A constant c in an assignment command of P satisfies $c \geq 2^w$.
2. Some input number satisfies $x[i] \geq 2^w$.

The Word-RAM Model

Definition: crashing

A program P crashes on input x and word length w if:

1. A constant c in an assignment command of P satisfies $c \geq 2^w$.
2. Some input number satisfies $x[i] \geq 2^w$.
3. The memory size satisfies $S > 2^w$.

The Word-RAM Model

Definition: crashing

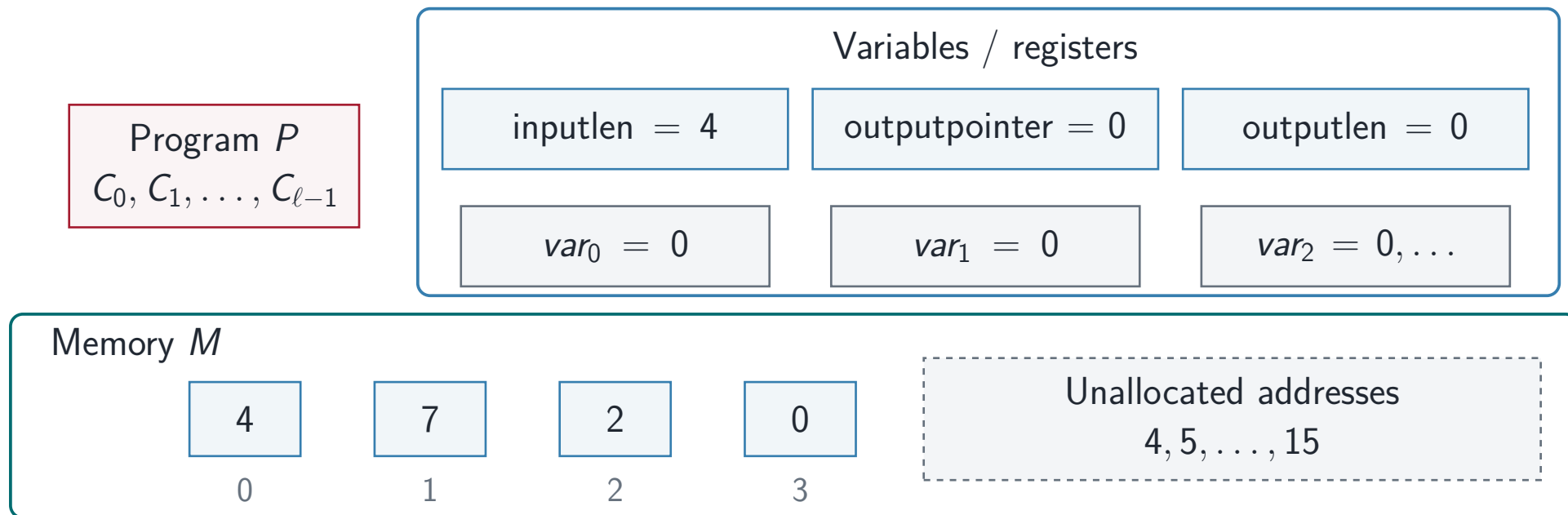
A program P crashes on input x and word length w if:

1. A constant c in an assignment command of P satisfies $c \geq 2^w$.
2. Some input number satisfies $x[i] \geq 2^w$.
3. The memory size satisfies $S > 2^w$.

The last case can occur initially, or after a MALLOC command. Arithmetic saturation itself does not cause a crash.

The Word-RAM Model

Example: $w = 4$, $S = 4$



The Word-RAM Model

Executions and example programs

Write $P[w](x)$ for the computation with word length w . It can halt with an output, fail to halt, or crash.

The Word-RAM Model

Executions and example programs

Write $P[w](x)$ for the computation with word length w . It can halt with an output, fail to halt, or crash.

$T_{P[w]}(x)$ counts commands until halting or crashing, and is ∞ if the execution never terminates.

The Word-RAM Model

Word length as a parameter

Q: What is wrong with just fixing $w = 64$ in the definition and using it as our model of computation?

The Word-RAM Model

Word length as a parameter

Q: What is wrong with just fixing $w = 64$ in the definition and using it as our model of computation?

There would be at most 2^{64} addressable memory cells, each holding 64 bits. The machine could not handle arbitrarily large inputs.

The Word-RAM Model

Word length as a parameter

Q: What is wrong with just fixing $w = 64$ in the definition and using it as our model of computation?

There would be at most 2^{64} addressable memory cells, each holding 64 bits. The machine could not handle arbitrarily large inputs.

An algorithm is a fixed procedure that works on arbitrarily large inputs. The same Word-RAM program can therefore run at different word lengths.

The Word-RAM Model

Word length as a parameter

Q: What is wrong with just fixing $w = 64$ in the definition and using it as our model of computation?

There would be at most 2^{64} addressable memory cells, each holding 64 bits. The machine could not handle arbitrarily large inputs.

An algorithm is a fixed procedure that works on arbitrarily large inputs. The same Word-RAM program can therefore run at different word lengths.

The word length may change *between* runs, but remains fixed *during* each run.

The Word-RAM Model

Definition: solving a computational problem

P solves $\Pi = (\mathcal{I}, \mathcal{O}, f)$ if, for every $x \in \mathcal{I}$:

The Word-RAM Model

Definition: solving a computational problem

P solves $\Pi = (\mathcal{I}, \mathcal{O}, f)$ if, for every $x \in \mathcal{I}$:

1. For every w , $P[w](x)$ terminates, producing an output or crashing.

The Word-RAM Model

Definition: solving a computational problem

P solves $\Pi = (\mathcal{I}, \mathcal{O}, f)$ if, for every $x \in \mathcal{I}$:

1. For every w , $P[w](x)$ terminates, producing an output or crashing.
2. For all sufficiently large w , $P[w](x)$ does not crash.

The Word-RAM Model

Definition: solving a computational problem

P solves $\Pi = (\mathcal{I}, \mathcal{O}, f)$ if, for every $x \in \mathcal{I}$:

1. For every w , $P[w](x)$ terminates, producing an output or crashing.
2. For all sufficiently large w , $P[w](x)$ does not crash.
3. For every w on which it does not crash,

$$P[w](x) \in f(x) \quad \text{if } f(x) \neq \emptyset, \quad P[w](x) = \perp \quad \text{if } f(x) = \emptyset.$$

The Word-RAM Model

Definition: running time

The running time of P on x is

The Word-RAM Model

Definition: running time

The running time of P on x is

$$T_P(x) = \max_{w \in \mathbb{N}} T_{P[w]}(x),$$

with value ∞ if these times are unbounded.

The Word-RAM Model

Definition: running time

The running time of P on x is

$$T_P(x) = \max_{w \in \mathbb{N}} T_{P[w]}(x),$$

with value ∞ if these times are unbounded.

As before, worst-case running time is

$$T_P(n) = \max_{\text{size}(x) \leq n} T_P(x).$$

The Word-RAM Model

Definition: running time

The running time of P on x is

$$T_P(x) = \max_{w \in \mathbb{N}} T_{P[w]}(x),$$

with value ∞ if these times are unbounded.

As before, worst-case running time is

$$T_P(n) = \max_{\text{size}(x) \leq n} T_P(x).$$

This measures performance across word lengths, including runs that crash.